김수환 포트폴리오

프로젝트별 담당 업무와 문제 해결 과정, 구현 구조와 실제 화면을 함께 정리했습니다.

Bitria 암호화폐 브로커리지 플랫폼

2025.11 – 현재 · 운영 중　웹·iOS·Android·데스크톱 프론트엔드 전담(앱 스토어 출시)·백엔드 API 개발 · 2인 팀

외부 거래소 유동성을 연동해 현물·선물·마진 거래를 제공하는 암호화폐 브로커리지 플랫폼입니다.
2인 팀에서 웹·iOS·Android·데스크톱 프론트엔드 전체를 초기 구축부터 개발했습니다.
주문·인증·자산 API 개발과 인프라 운영에도 참여했습니다.

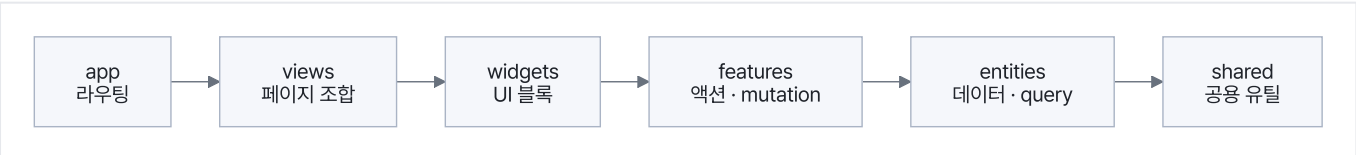
주요 업무

공통 구조	FSD(Feature-Sliced Design) 구조와 시맨틱 디자인 토큰 설계, 다크·라이트 테마·브랜드 2종·6개 로케일 지원, 웹·앱별 코드 분리 및 컴포넌트 API 기준 문서화
거래 화면	TradingView 차트, 실시간 시세·호가·체결, 현물·선물·마진 주문 및 포지션·체결 내역 구현
빌드·성능	Next.js 16.4·Turbopack 전환, 번들·폰트 최적화, 실시간 화면 리렌더 격리
멀티 플랫폼	하나의 코드베이스로 웹·iOS·Android·Electron 앱 빌드, 모바일 앱 스토어 출시, 네이티브 푸시·소셜 로그인·생체인증 연동
인증·보안	회원가입·소셜 로그인, 2단계 인증(2FA)·패스키·신뢰 기기·세션·계정 복구 개발, 고객확인(KYC)·입출금 연동 및 API 키 관리 구현
백엔드 API	주문 생성·조회·취소·수정 API 개발 및 요청 검증·권한 확인·주문 처리 서비스 연동, 계좌별 자산·USDT 환산·스냅샷·일별 손익 조회 구현
서비스 운영	가격 조건별 알림·클라이언트 오류 수집·앱 버전 정책·고객 문의(CS) API 개발, K3s 프로덕션 배포·운영 및 장애 대응
AI 개발 환경	AI 코딩 에이전트 도입, 아키텍처·디자인 토큰·로깅 규칙을 에이전트 규칙 문서로 관리, 여러 에이전트 세션에 작업을 나눠 병렬 진행, 성능 계측·버그 재현 자동화

구현 구조와 설계

FSD 아키텍처와 화면 분기 기준

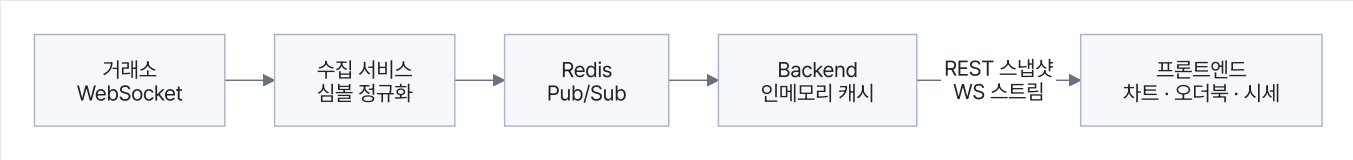
화면과 도메인이 늘어나도 의존성이 뒤섞이지 않도록 app → views → widgets → features → entities → shared 순의 단방향 규칙을 정했습니다.
조회(query)는 entity에, 변경(mutation)은 feature에 두어 각 레이어의 책임을 분리했습니다.
웹과 앱의 라우트가 다르면 폴더를 분리했습니다.
화면 크기 차이가 작으면 prop으로 분기하고, 마크업 차이가 크면 디바이스별 파일을 두도록 기준을 정했습니다.



FSD 레이어 간 의존 방향과 각 레이어의 책임

실시간 데이터 전달 구조

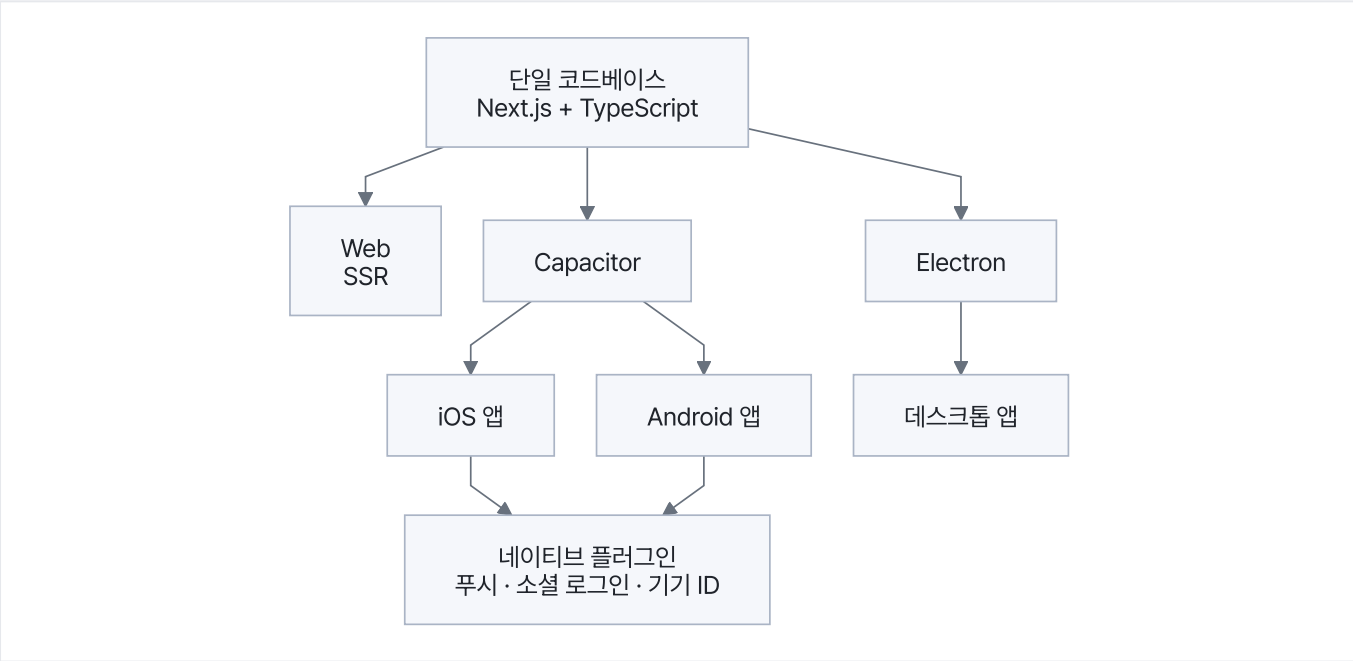
거래소 WebSocket 데이터는 수집 서비스의 심볼 정규화와 Redis Pub/Sub을 거쳐 백엔드에 전달됩니다.
프론트엔드는 REST 스냅샷과 WebSocket 스트림으로 차트·호가·시세를 표시합니다.



실시간 데이터 전달 경로

단일 코드베이스의 멀티 플랫폼 대응

2인 팀에서 웹·iOS·Android·데스크톱을 빠르게 함께 내기 위해, 각 플랫폼을 따로 만들지 않고 기존 Next.js 코드베이스를 그대로 쓰는 Capacitor와 Electron을 선택했습니다.
Next.js·TypeScript 코드베이스를 웹, Capacitor 기반 iOS·Android 앱, Electron 데스크톱 앱으로 빌드했습니다.
모바일 앱을 스토어에 출시하고 푸시(APNs·FCM)·소셜 로그인·기기 식별 기능을 연결했습니다.
앱 강제 업데이트 정책을 연동하고, Electron 앱의 API 요청을 식별하는 헤더를 추가했습니다.



웹·모바일·데스크톱 빌드와 네이티브 기능 연결 구조

문제 해결과 개선

배럴 import의 불필요한 모듈 유입 개선

공통 컴포넌트를 한 경로로 가져오도록 만든 shared/ui 배럴이 하위 배럴 10개를 다시 내보내고, 그중에는 three.js 기반 3D 컴포넌트도 있었습니다.
번들 분석 결과 버튼 하나를 import해도 Radix 전체와 three.js까지 163개 모듈이 하나의 청크로 묶여 평가됐고, 이 청크가 모바일에서 가장 오래 실행되는 청크였습니다.
배럴 import 136곳을 컴포넌트 정의 파일 직접 참조로 바꿔 이 청크를 없앴습니다.

optimizePackageImports도 비교했지만 청크 수와 전체 번들이 증가해 채택하지 않았습니다.
three.js 기반 장식용 배경은 별도 조치로 next/dynamic을 이용해 클라이언트 청크로 분리했습니다.

> 직접 import 변경 전후의 로컬 프로덕션 빌드를 각 5회 측정한 결과, 모바일 Lighthouse TBT 중앙값은 417ms에서 292.5ms로 감소했습니다.

측정 조건은 로컬 프로덕션 빌드, 모바일 CPU 4배 감속, 네트워크 제한입니다.
three.js 동적 import는 전후 모두 적용돼 있어 직접 import 변경만 비교했습니다.
외부 API 응답과 측정 당시 시스템 부하에 따라 수치는 달라질 수 있습니다.

빌드·번들 최적화

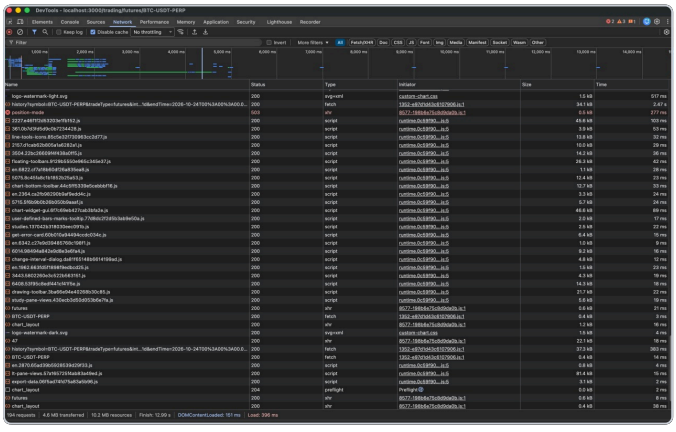
SVG를 React 컴포넌트로 쓰는 곳과 이미지 주소(`?url`)로 쓰는 곳 45곳이 섞여 있었습니다.
webpack은 import 쿼리로 둘을 나눌 수 있었지만 당시 Turbopack 규칙은 쿼리 분기를 지원하지 않아, 개발 서버는 Turbopack, 프로덕션 빌드는 webpack으로 나눠 운영했습니다.
그래서 개발 서버에서는 `?url` 아이콘이 표시되지 않는 차이도 남아 있었습니다.

Next.js 16.2부터 Turbopack 규칙에 쿼리 조건이 추가된 것을 확인하고 16.4로 올려 webpack과 같은 SVG 분기를 적용했습니다.
빌드를 Turbopack으로 통일하고 webpack 설정은 폴백으로 남겼습니다.
전환 중 상위 모노레포의 lockfile 때문에 Tailwind가 프로젝트 밖 경로까지 탐지해 빌드가 실패하던 문제는 소스 경로를 명시해 해결했습니다.

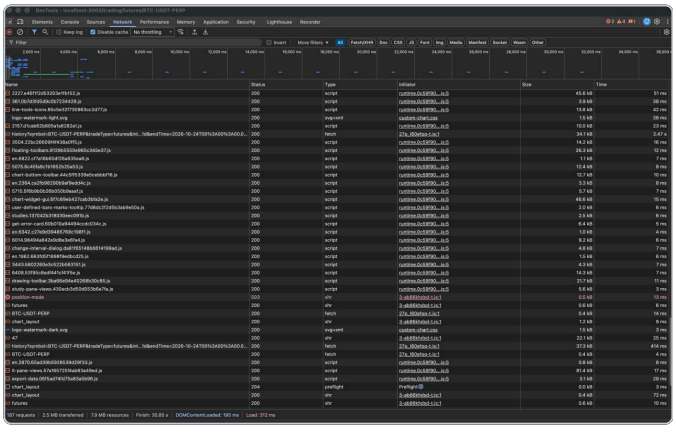
부수효과 선언이 없으면 번들러는 모든 모듈에 부수효과가 있다고 보고 쓰지 않는 모듈도 평가합니다.
다만 소켓 싱글턴·API 인터셉터·오류 수집처럼 불러오는 순간 동작해야 하는 모듈 4개가 있어 전부 제외할 수는 없었고, 이 4개와 CSS만 예외로 명시했습니다.
한국어 화면을 지원하지 않는데도 한글 11,172자가 담긴 Pretendard 전체 폰트를 모든 화면에서 받고 있어, 화면에 필요한 글자 구간만 받는 서브셋으로 바꿨습니다.

> 프로덕션 빌드 시간은 74.2초에서 32.8초로 56%, 홈 JS는 752KB에서 550KB로 27%, 초기 폰트 전송은 2,048KB에서 111KB로 95% 줄었습니다.

빌드 시간은 캐시 없는 프로덕션 빌드를 전후 커밋 교대로 3회 측정한 중앙값입니다.
JS 크기는 프로덕션 빌드 결과물의 gzip 기준, 폰트 전송량은 홈 화면 기준입니다.



개선 전: 거래 화면 콜드 로드 4.6MB 전송 · 194 요청



개선 후: 같은 조건에서 2.5MB 전송 · 187 요청

보이지 않는 화면이 렌더 시간을 차지하던 문제 해결

앱 거래 화면에서 차트 아이콘을 누르면 별도 차트 화면으로 이동했고, 그때부터 캔들 조회와 소켓 연결을 시작해 빈 화면을 기다려야 했습니다.
주문 화면과 차트 화면을 함께 마운트해 두고 보이는 쪽만 전환해, 숨겨진 차트가 캔들과 소켓을 미리 준비하도록 바꿨습니다.
display:none으로 숨기면 크기가 0이라 TradingView가 초기화되지 않아, visibility로 숨겨 실제 크기를 유지했습니다.

그 대가로 숨겨진 화면도 시세가 바뀔 때마다 다시 렌더링됐습니다.
React가 렌더할 때마다 알려주는 내부 연결 지점(DevTools 훅)에 측정 코드를 붙여 컴포넌트별 렌더 횟수와 시간을 자동으로 수집했고, 숨겨진 차트 화면과 닫힌 오더북 드로어가 렌더 시간의 64%(1,028ms 중 653ms)를 차지하는 것을 확인했습니다.

두 화면은 시세 스냅샷을 각자 최상위 상태로 받고 있었고, 숨겨진 화면에 구독 비활성화 옵션을 넘겨도 시세 핸들러가 전역 목록으로 호출돼 상태 갱신이 멈추지 않았습니다.
실시간 값을 표시하는 컴포넌트가 직접 구독하도록 분리하고, 닫힌 드로어와 탭 뒤의 오더북은 구독을 멈췄습니다.

> 동일한 시세 메시지를 12초간 재생한 실험에서 렌더 작업 합계 중앙값이 5,192회에서 1,534회로 70% 줄었습니다.
> 차트를 미리 준비하는 동작은 그대로 유지하면서, 주문 폼·하단 패널·차트 헤더는 시세 갱신 중 다시 렌더링되지 않습니다.

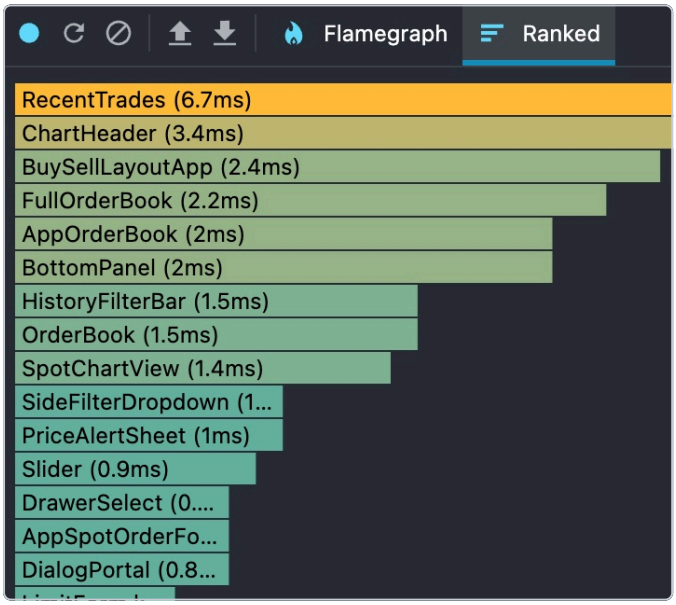
원인 진단 수치(64%)는 개발 빌드에서 실시간 갱신만 12초간 측정한 1회 결과입니다.
개선 결과는 개발 빌드, 모바일 화면(390×844), 같은 공개 시세 메시지 73개 재생, 개선 전후 각 5회 중앙값이며, 기기 부하 차이로 렌더 시간 대신 렌더 횟수를 대표 지표로 사용했습니다.

```
═══ 리렌더 측정 요약 (5회 중앙값) ═══
  입력 : 녹화한 시세 이벤트 73개를 12초 동안 동일하게 재생 (fixture sha256 7d33b33cdb8e…)
  계측 : React DevTools 훅(onCommitFiberRoot)으로 커밋마다 실제 렌더된 컴포넌트(PerformedWork) 집계

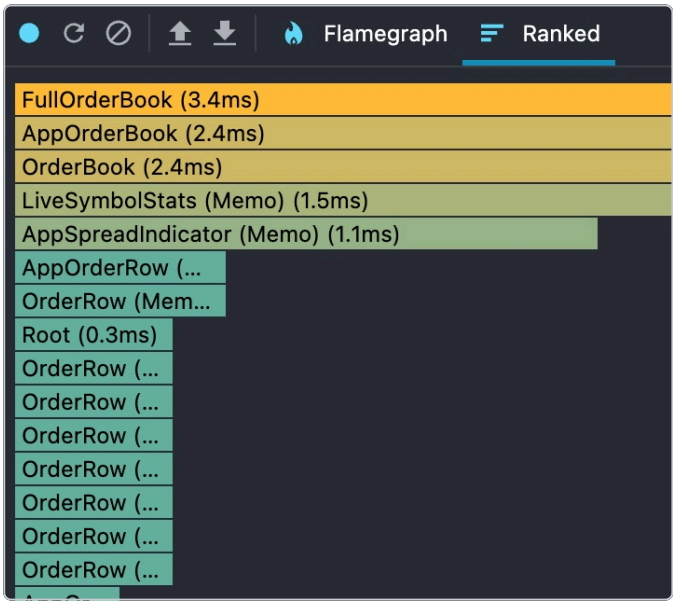
  지표                          개선 전          개선 후    변화
  렌더된 컴포넌트 합계          5,192회  →      1,534회    -70.5%
  React 커밋 수                    93회  →         75회    -19.4%
  React 렌더 시간                661.0ms  →      394.7ms    -40.3%

  컴포넌트별 렌더 횟수 (중앙값)
  BuySellLayoutApp                 8회  →          0회    -100.0%
  SpotChartView                   14회  →          0회    -100.0%
  ChartHeader                     14회  →          0회    -100.0%
  AppOrderForm                     8회  →          0회    -100.0%
  BottomPanel                      8회  →          0회    -100.0%
  AppOrderBook                    34회  →         28회    -17.6%
  RecentTrades                    32회  →         18회    -43.8%
```

측정 스크립트 실행 결과 · 렌더된 컴포넌트 합계와 컴포넌트별 렌더 횟수 (5회 중앙값)



개선 전 · 시세 갱신 한 번에 레이아웃·주문 폼까지 리렌더



개선 후 · 같은 갱신에 값 표시 컴포넌트만 리렌더

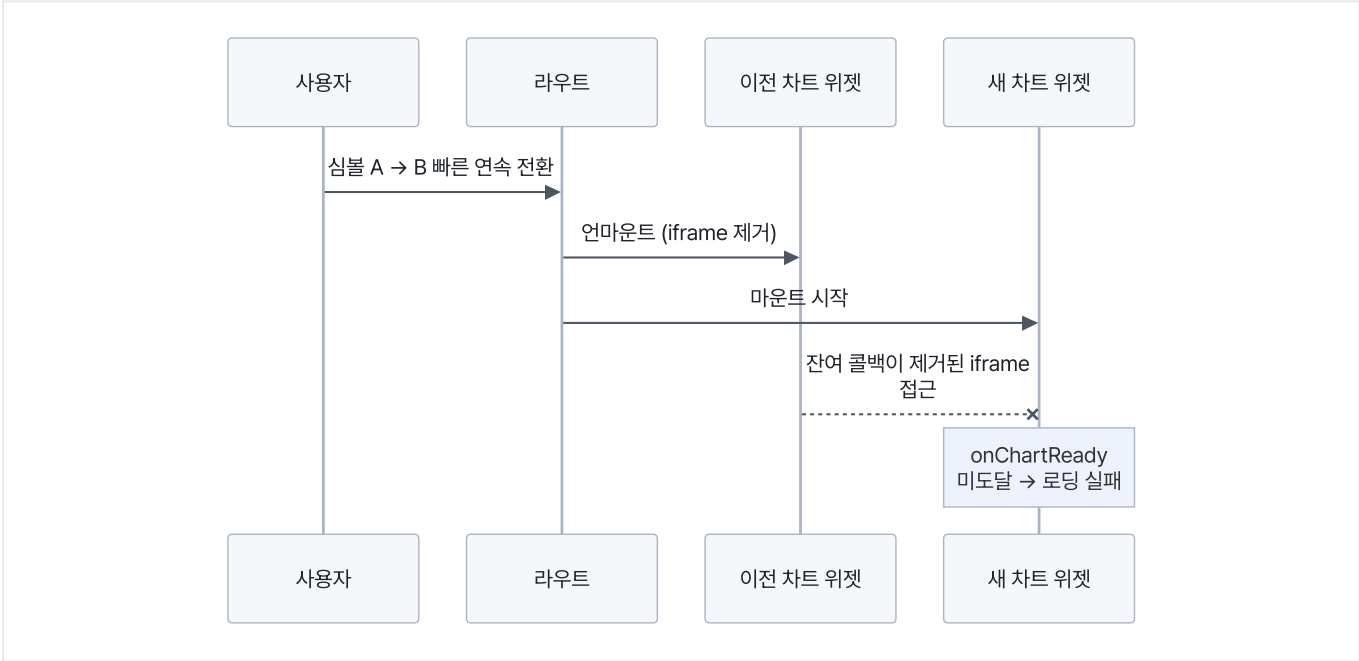
차트 로딩 실패 원인 추적과 해결

차트 로딩이 실패하면 로딩 화면에 그대로 멈춰 있었고, 실패 기록이 서버에 한 건도 남지 않아 원인을 알 수 없었습니다.
TradingView는 로드 실패 신호를 주지 않고, 스크립트는 실패 처리 없이 불러오고 있었으며, 데이터 요청에는 타임아웃이 없어 응답이 없으면 계속 대기했기 때문입니다.
스크립트 로드 재시도, 응답 없이 멈춘 요청의 타임아웃, 준비 완료 신호를 기다리는 안전망을 추가하고 실패 단계와 원인을 서버 로그로 보고하도록 했습니다.
TradingView가 캔들 요청 실패 뒤 다시 요청하지 않는 것을 확인해 직접 재시도하고, 그래도 실패하면 재시도 버튼을 보여주도록 했습니다.

수집된 로그에서 13일간 기록된 50건 대부분이 앱 거래 화면에서 발생한 것을 확인했습니다.
당시 차트는 인앱 브라우저에서 표시되지 않던 문제 때문에 호환 로딩 모드로 불러오고 있었고, Cloudflare가 정적 번들을 매번 원본 서버에서 다시 받던 문제를 고치려고 번들을 1년 캐시하도록 바꾼 상태였습니다.
TradingView 릴리스 노트에서 호환 로딩 모드와 캐시된 번들이 겹치면 준비 완료 신호가 누락되는 알려진 버그를 찾았고, 두 조건이 모두 일치했습니다.
라이선스 제약으로 수정된 버전으로 올릴 수 없어, 같은 출처(same_origin) iframe 로딩 방식으로 전환했습니다.
이 방식은 blob 주소를 쓰지 않아 인앱 브라우저 대응이라는 원래 목적과도 맞습니다.

남은 실패는 심볼을 빠르게 전환하는 사용자에게 몰려 있었고, 네트워크·서버 응답은 정상이었습니다.
운영 로그에는 타임아웃과 같은 시각에 이전 위젯이 제거된 iframe에 접근하다 난 오류(`applyOverrides` null 참조)가 함께 남아 있었습니다.
Next.js가 심볼 변경마다 페이지를 다시 마운트하면서 위젯 제거와 재생성이 겹친 것으로, WebKit에서 빠른 전환을 반복해 운영과 같은 실패 보고까지 재현했습니다.
거래 화면을 layout으로 옮겨 위젯을 다시 만들지 않고 심볼만 교체하도록 했고, 준비 전에 들어온 심볼 전환은 보관했다가 반영하도록 했습니다.
숨겨진 차트가 실패로 집계되던 오판을 없애고, 스크립트 로드 실패는 타임아웃 대신 리소스 오류 이벤트로 즉시 감지하도록 바꿨습니다.

> 빠르게 심볼을 전환해도 마지막에 선택한 심볼로 차트가 정상 로딩되고, 실패하더라도 로딩 화면에 갇히지 않고 재시도로 복구됩니다.
> 실패는 단계별 원인과 함께 운영 로그에 남고, 정상 로드에서는 실패로 오판하지 않는 것을 함께 확인했습니다.



수정 전 심볼 전환 경쟁 상태: 이전 위젯에 남아 있던 콜백이 제거된 iframe에 접근해 새 위젯이 준비 완료에 도달하지 못하는 흐름

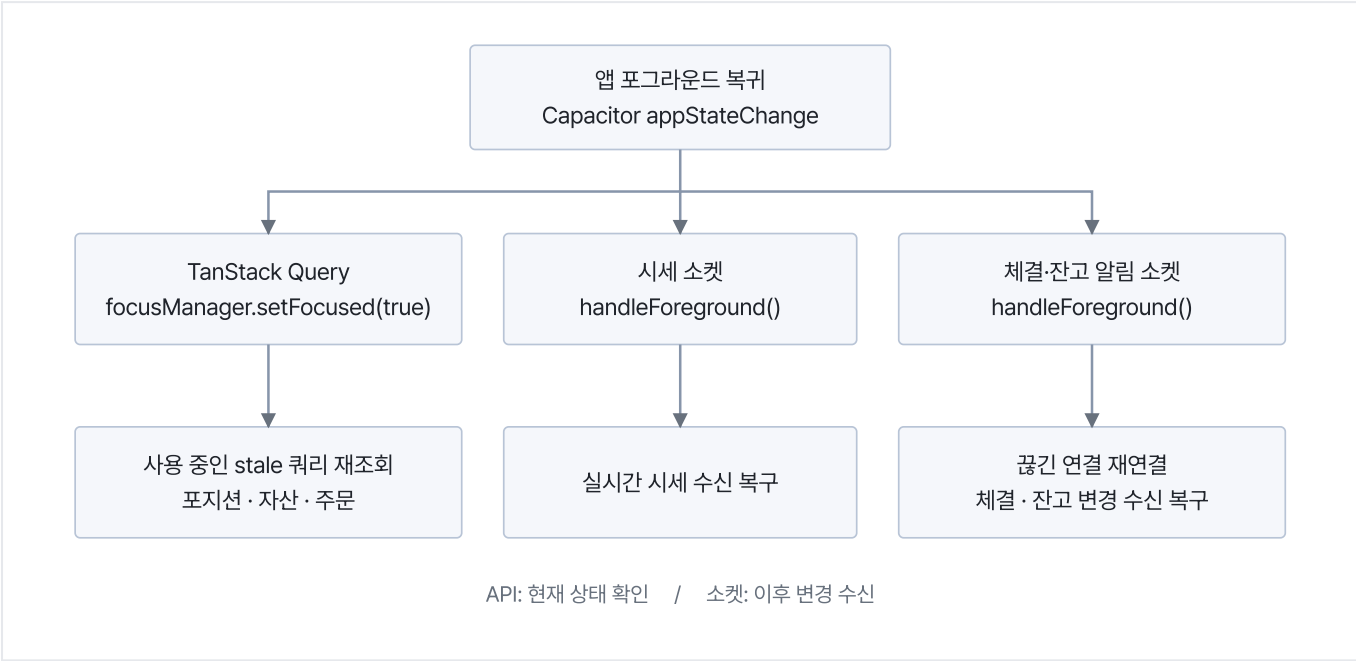
앱 복귀 후 포지션·자산 정보 불일치 해결

iOS 앱을 백그라운드에 두었다가 복귀하면 이미 종료된 포지션이 화면에 그대로 남고, 현재 시세로 계산한 손익까지 표시되는 문제가 있었습니다.
시세 연결은 복구되지만 포지션 조회 캐시는 갱신되지 않아, 다음 정기 조회까지 최대 2분간 오래된 정보가 남았습니다.

원인은 두 가지였습니다.
iOS WKWebView는 백그라운드에서 복귀할 때 웹의 visibilitychange 이벤트를 보내지 않아 TanStack Query가 복귀를 감지하지 못했습니다.
알림 소켓은 백그라운드에 있는 동안 재연결 5회 제한을 모두 써버려 재연결을 포기했습니다.
같은 제한 문제를 앞서 시세 소켓에서만 고쳤고, 네이티브 앱 복귀 이벤트도 시세 소켓에만 연결돼 있었습니다.

Capacitor의 appStateChange를 TanStack Query focusManager에 연결해 사용 중인 stale 쿼리를 다시 조회하도록 했습니다.
알림 소켓에는 복귀 시 명시적 재연결과 지속 재시도를 추가했습니다.
현재 상태를 확인하는 API 재조회와 이후 변경을 받는 소켓 연결을 함께 복구하도록 구성했습니다.

앱 복귀 시 종료된 포지션이 남지 않고, 포지션·자산이 바로 최신 상태로 표시됩니다.
페이지 새로고침 없이 갱신이 필요한 쿼리만 다시 요청합니다.



앱 복귀 이벤트에서 현재 상태 재조회와 실시간 연결 복구를 함께 실행하는 흐름

비영어 기기에서 앱 대신 Safari가 열리던 문제 해결

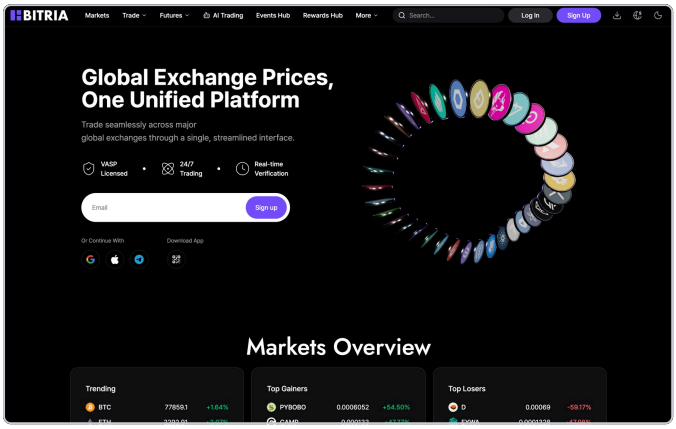
일본어·스페인어·베트남어·태국어·중국어 기기에서 iOS 앱을 실행하면 앱 화면 대신 Safari로 사이트가 열리는 문제가 있었습니다.
영어 이외의 지원 언어를 쓰는 기기에서 공통으로 발생했습니다.

Bitria 앱은 서버 렌더링과 앱 전용 경로로 보내는 미들웨어 등 Next.js 서버 기능을 그대로 쓰기 위해 Capacitor가 원격 URL(server.url)로 웹을 불러오는 방식이고, 언어 설정은 기본 언어인 영어 주소를 깔끔하게 유지하려고 영어에만 언어 코드를 붙이지 않는 as-needed 방식이었습니다.
Capacitor iOS 소스를 확인한 결과, 앱 내부 이동 여부를 server.url과의 문자열 접두사 비교로 판정하고 있었습니다.
next-intl의 as-needed 설정으로 기본 언어 외에는 /th/m처럼 주소 앞에 언어 코드가 붙어, server.url(/m)과 접두사가 달라지면서 외부 링크로 판정됐습니다.
두 설정은 각각 정상이었지만 함께 쓰일 때 문제가 생겼습니다.

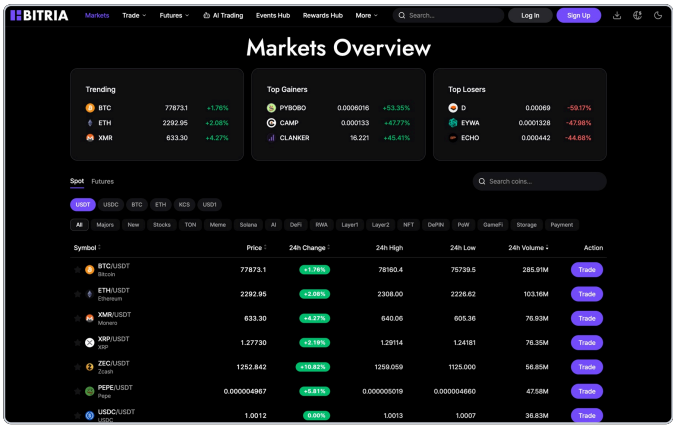
앱 요청에 한해 미들웨어에서 언어 코드를 붙이는 리다이렉트를 같은 대상의 rewrite로 바꿨습니다.
주소는 /m으로 시작하도록 유지하면서 화면은 기기 언어로 표시하고, 웹 동작은 바꾸지 않았습니다.

영어 외 지원 언어 5개(일본어·스페인어·베트남어·태국어·중국어) 기기에서 앱이 정상 실행됩니다.
원격 URL 방식이라 앱 재빌드·스토어 재심사 없이 서버 배포만으로 적용했습니다.

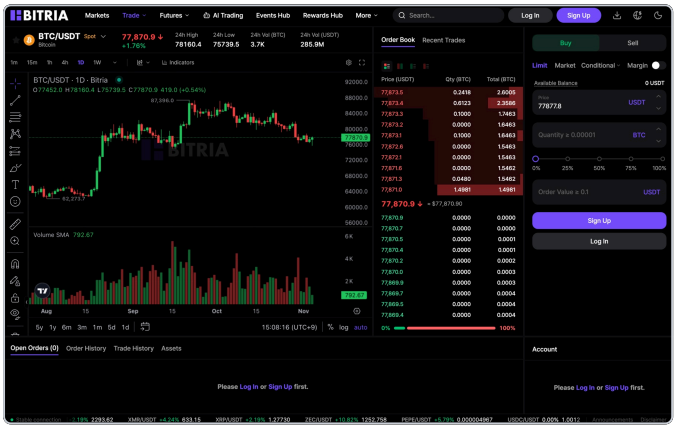
Bitria · 프로젝트 화면



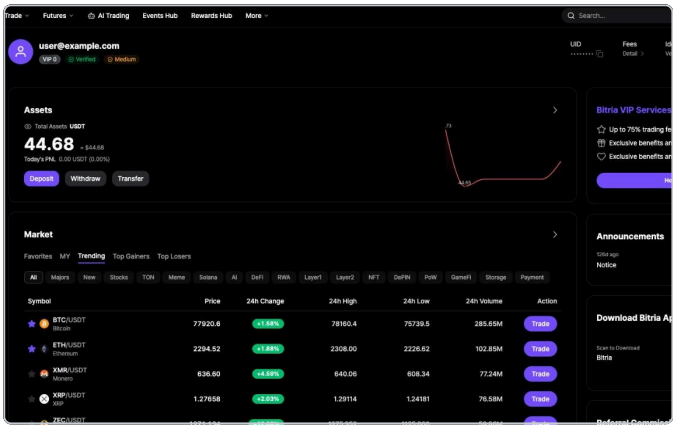
웹 홈 화면



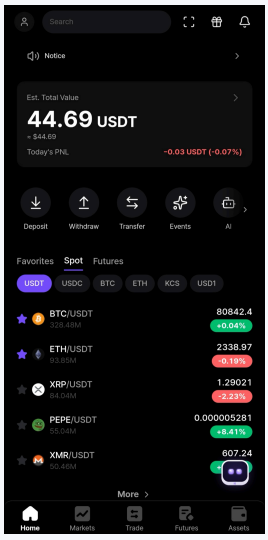
웹 마켓 화면



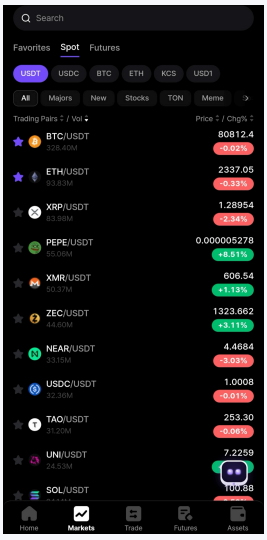
웹 거래 화면



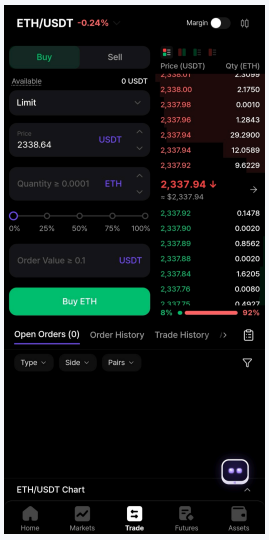
웹 마이페이지 화면 (이메일·UID 가림)



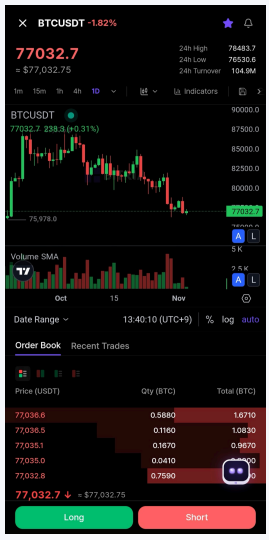
앱 홈 화면



앱 마켓 화면



앱 현물 거래 화면



앱 선물 차트 화면

기술 스택

프론트엔드	TypeScript · Next.js · Tailwind CSS · next-intl · Zustand · TanStack Query · Capacitor · Electron
백엔드	Node.js · Express · Prisma · MariaDB · TimescaleDB · Redis · RabbitMQ · Socket.IO · AWS(EC2) · Docker · Kubernetes(K3s)

Gobit 코인 정보·커뮤니티 플랫폼

2024.10 – 2025.12　프론트엔드·백엔드 개발 담당 · 2인 팀

암호화폐 시세·차트·뉴스·커뮤니티와 모의투자 기능을 제공하는 코인 정보 플랫폼입니다.
다른 개발자가 만든 기존 코드가 유실돼, 2인 팀에서 Next.js 웹 클라이언트를 새로 구축하고 Node.js·Express API를 개발했습니다.
Binance 시세를 수집해 Redis에 캐시하고 Socket.IO로 전달했으며, 회원·게시글 기능과 운영 도구를 구현했습니다.

주요 업무

웹 클라이언트	Next.js 15·React 19 기반 마켓·코인 상세 등 주요 화면 구현, TradingView 차트·tiptap 에디터 연동
실시간 데이터	시세는 Socket.IO, 나머지 조회 데이터는 SWR로 관리, Binance 시세 Redis 캐시 및 소켓 재연결·헬스체크 개선
서비스 기능	회원 인증·출석·레퍼럴·게시글·댓글·북마크 기능과 관련 API 개발
운영 개선	SSR·이미지·폰트 최적화로 SEO·코인 상세 페이지 LCP 개선, coinIns → Gobit 리브랜딩 및 점검 모드·회원 탈퇴 구현

구현 구조와 설계

실시간 시세와 일반 조회 데이터의 분리

Binance WebSocket으로 수집한 시세를 Redis에 캐시하고 Socket.IO로 웹 클라이언트에 전달했습니다.
시세 이외의 조회 데이터는 SWR로 관리해 실시간 스트림과 API 조회의 역할을 구분했습니다.
TradingView 차트와 tiptap 에디터를 연동하고 회원 인증·게시글·댓글·북마크 기능을 구현했습니다.

레거시 서비스의 통합·배포 구성

다른 개발자가 만든 레거시 서비스는 소스 코드가 유실돼 바이너리로만 서버 5대에 흩어져 있었습니다.
Gobit·모의투자·코인 매매 시그널 서비스를 모노레포로 통합했습니다.
수집기 → Redis → 서버로 데이터 전달 경로를 정리하고, 서비스별 Dockerfile과 Kubernetes 매니페스트를 구성했습니다.
GitHub Actions로 빌드·배포를 자동화하고, 설정값은 Kubernetes Secret에서 불러오도록 구성했습니다.
DB 백업과 캔들 보관 작업은 CronJob으로 분리했습니다.

문제 해결과 개선

레거시 플랫폼 통합과 Kubernetes 이전

2026.07 – 2026.08 · 이전 후 유지보수

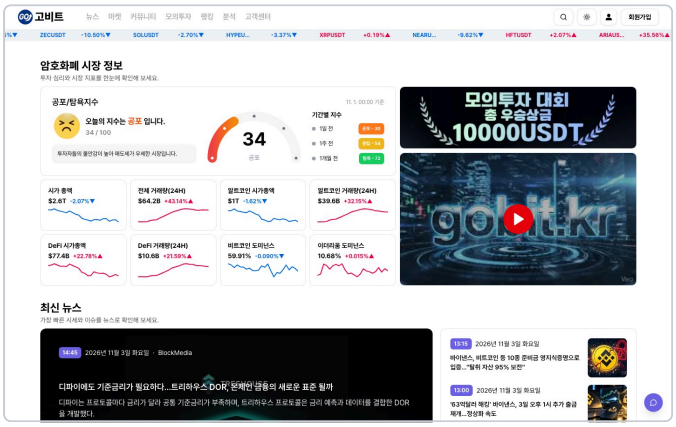
Gobit과 다른 개발자가 만든 모의투자 시스템·코인 매매 시그널 서비스가 Windows·Linux 서버 5대에 흩어져 있었습니다.
레거시 서비스는 소스 코드가 유실돼 핵심 시세 수집기는 .NET 바이너리만 남아 있었습니다.
유지보수와 배포를 이어갈 수 있도록 서비스를 모노레포로 통합하고 Kubernetes 환경으로 이전했습니다.

- 기존 바이너리의 동작과 데이터 형식을 분석해 수집기를 Node.js로 재작성했습니다.
  실시간 시세·캔들·과거 데이터 수집 모드를 구현하고, 연결 분산 처리와 요청 속도 제한으로 WebSocket 연결 끊김에 대응했습니다.
- 중계 계층을 정리해 데이터 흐름을 수집기 → Redis → 서버로 단순화했습니다.
- 13개 서비스의 Dockerfile과 Kubernetes 매니페스트를 작성하고, Ingress 설정 및 DB 백업·캔들 보관 CronJob을 구성했습니다.

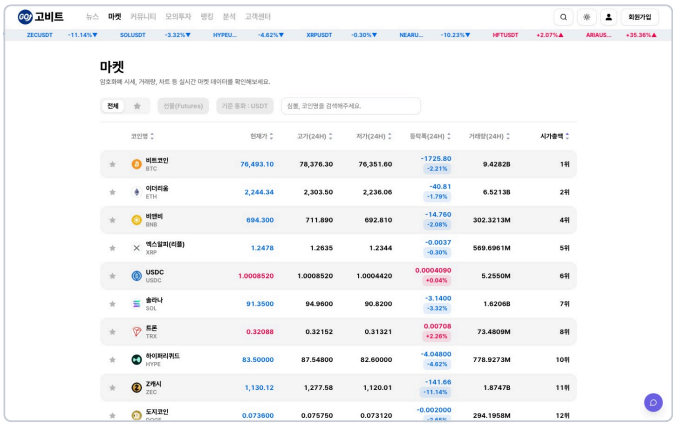
- GitHub Actions 빌드·배포를 자동화하고, 설정값을 Kubernetes Secret에서 불러오도록 변경했습니다.

> 서버 5대에 흩어져 있던 13개 서비스를 하나의 Kubernetes 환경에서 배포·운영합니다.
> 소스 없이 바이너리만 남아 있던 수집기도 코드로 관리할 수 있게 됐습니다.

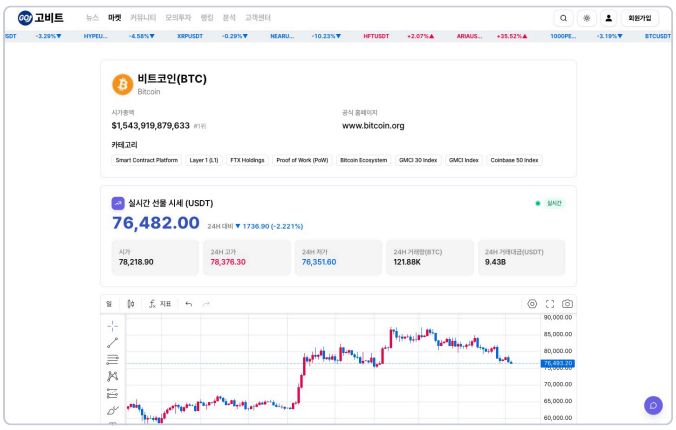
Gobit · 프로젝트 화면



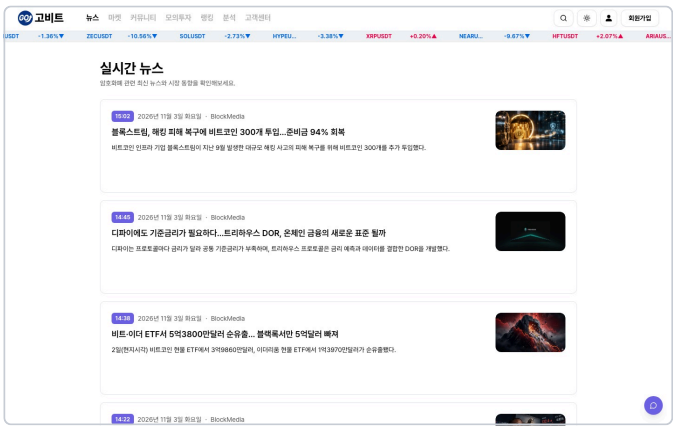
홈 화면



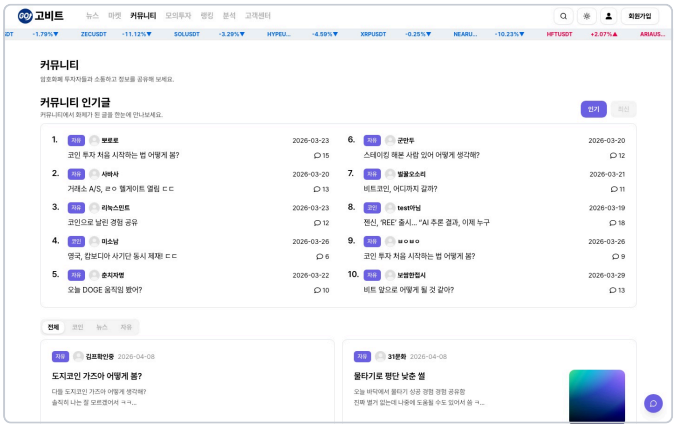
마켓 화면



코인 상세 화면



뉴스 화면



커뮤니티 화면

기술 스택

프론트엔드	TypeScript · Next.js · Tailwind CSS
백엔드	Node.js · Express · MariaDB · Redis · Socket.IO

두잇 홈페이지 리뉴얼 회사 홈페이지 React 마이그레이션

2024.05 – 2024.07 프론트엔드 개발 담당

사내에 React를 도입하면서 회사 홈페이지도 React로 옮겨 새로 단장하기로 했습니다.
jQuery 기반 회사 홈페이지를 React·TypeScript·Vite SPA로 재작성해 한국어·영어와 반응형 화면을 지원했습니다.

주요 업무

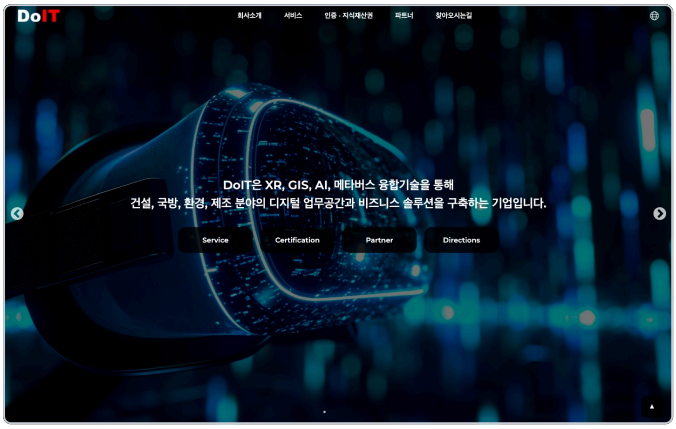
마이그레이션	메인·회사소개·서비스·인증·파트너 5개 페이지를 컴포넌트 구조로 재작성
UI 재사용	사내 UI 라이브러리 Doit UI Kit을 적용해 색상·타이포·레이아웃 등 공통 요소 재사용
다국어	i18next로 한국어·영어를 지원하고 번역 리소스를 페이지별 네임스페이스로 분리
반응형	CSS·JavaScript 브레이크포인트 통일, 화면 크기 변화는 CSS로 처리하고 모바일에서 불필요한 영역은 렌더링 제외

구현 구조와 설계

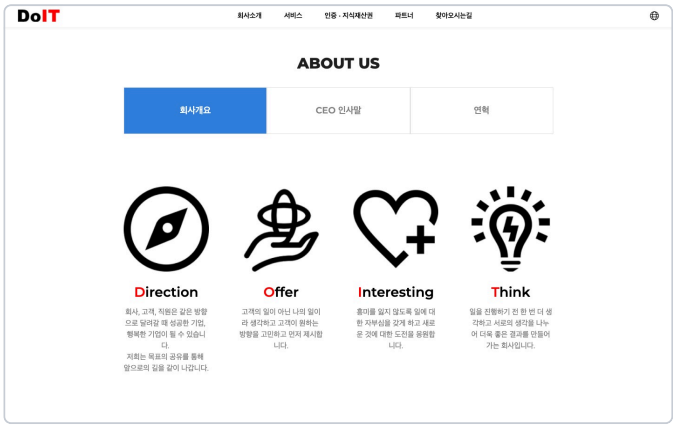
공통 UI와 페이지별 구성

색상·타이포·레이아웃 등의 공통 요소에 Doit UI Kit을 적용하고, 페이지별 콘텐츠를 React 컴포넌트로 구성했습니다.
번역 리소스는 i18next의 페이지별 네임스페이스로 분리했습니다.
CSS와 JavaScript에서 같은 브레이크포인트를 사용하고, 모바일에서 불필요한 영역은 렌더링하지 않도록 구성했습니다.

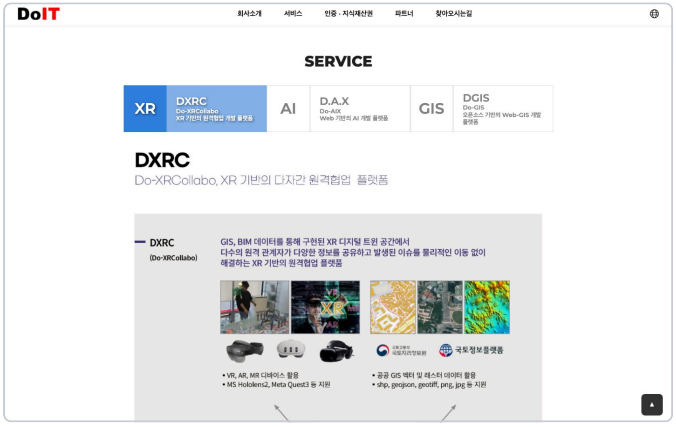
두잇 홈페이지 리뉴얼 · 프로젝트 화면



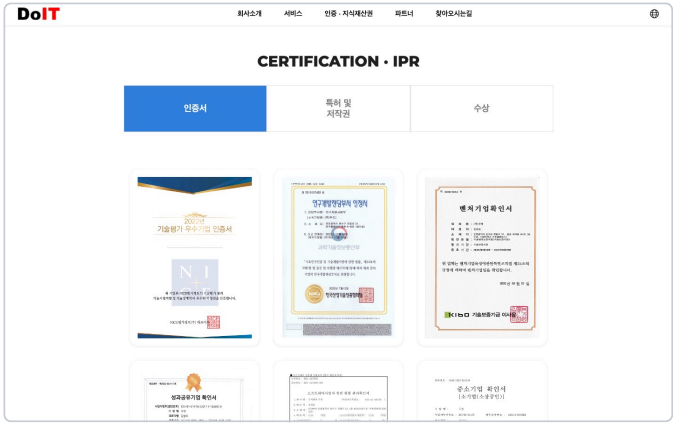
메인 화면



회사소개 화면



서비스 화면



인증 화면

기술 스택

프론트엔드 TypeScript · React · styled-components · i18next

Doit UI Kit 사내 공통 UI 라이브러리

2024.02 – 2024.07 프론트엔드 개발 담당

프로젝트마다 반복 구현하던 UI를 공통 라이브러리로 만들고, React 개발 규칙을 팀에 공유했습니다.

주요 업무

컴포넌트	Atomic Design 기반 UI 컴포넌트 24종 개발: 기본 컴포넌트 12종, 차트 5종을 포함한 복합 컴포넌트 12종
패키징	React·React DOM을 peer dependency로 분리하고 vite-plugin-dts로 타입 선언을 자동 생성해 npm 배포
문서화	Storybook·Chromatic으로 사용법·Props·상태별 UI 문서화
팀 표준	디렉토리 구조·디자인 패턴·라이브러리 선정 기준 정립, ESLint·Prettier 설정 및 팀 온보딩 교육

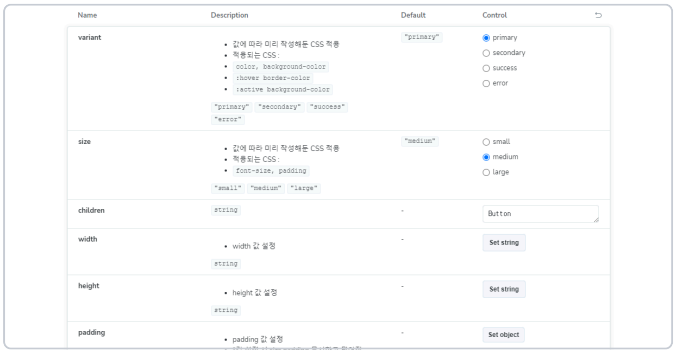
> D.A.X와 회사 홈페이지가 같은 컴포넌트와 개발 규칙을 사용하게 됐습니다.
> 두 프로젝트에서 나온 요구사항은 라이브러리에 반영해 다시 배포했습니다.

구현 구조와 설계

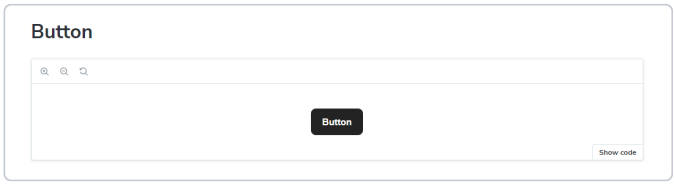
컴포넌트 조합과 라이브러리 배포

Button·Input 같은 Atom을 조합해 FileInput 등의 Molecule을 구성했습니다.
컴포넌트의 Props와 상태별 UI를 Storybook에 문서화하고, 사용하는 프로젝트에서 같은 기준으로 확인하도록 구성했습니다.
Playground에서는 값을 직접 바꿔 사용 예를 확인할 수 있습니다.
vite-plugin-dts로 타입 선언을 함께 배포하고 React·React DOM은 peer dependency로 분리했습니다.
D.A.X와 회사 홈페이지에 적용하면서 발견한 요구사항을 라이브러리에 반영했습니다.

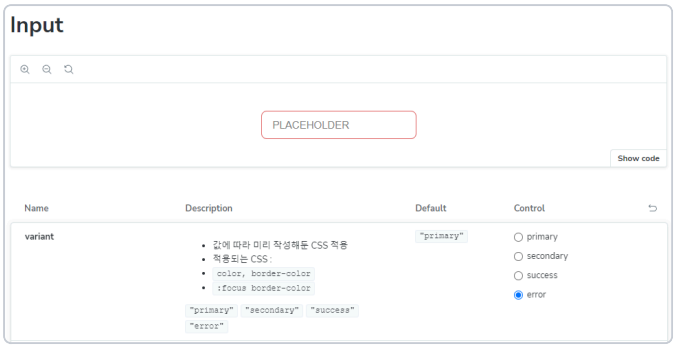
Doit UI Kit · 프로젝트 화면



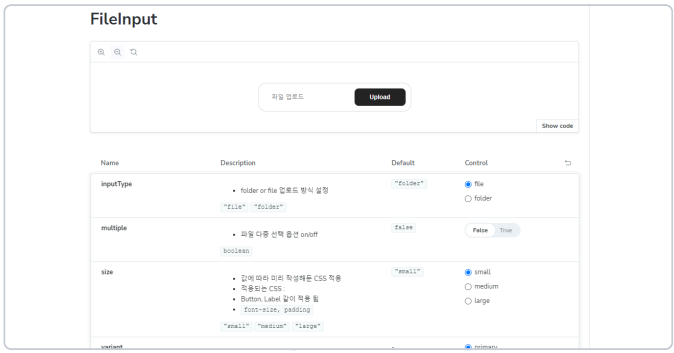
Doit UI Kit 컴포넌트의 Storybook 문서와 Props 설정 화면



Button 컴포넌트의 Storybook 예시



Input 컴포넌트의 상태·Props 문서



Input·Button·Label을 조합한 FileInput 컴포넌트

기술 스택

프론트엔드 TypeScript · React · styled-components · Storybook

D.A.X AI 모델 학습·운영 플랫폼

2024.01 – 2024.06　프론트엔드 개발 담당 · 백엔드 1명과 협업

데이터 수집·라벨링부터 모델 학습·추론까지 지원하는 사내 플랫폼입니다.
백엔드 개발자 1명과 협업해 프론트엔드 전 화면을 개발했습니다.

주요 업무

플랫폼 화면　프로젝트·대시보드·클래스·컬렉션·라벨링·모델 학습·추론 화면 구현

데이터 도구　캔버스 기반 바운딩 박스 라벨링, 이미지·폴더 업로드, 컬렉션 ZIP 다운로드 기능 개발

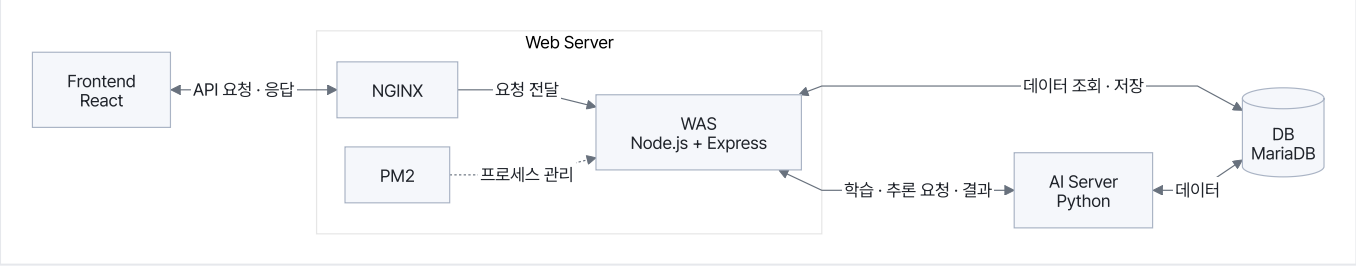
공통 구조　검색 조건·페이지네이션·로딩 상태 처리, 사내 표준 구조와 Doit UI Kit 적용

> 라벨링 이미지는 화면 크기에 맞춰 축소되지만 학습용 좌표는 원본 해상도로 저장해야 했습니다.
> 원본↔화면 좌표의 양방향 변환을 전용 훅으로 분리해 그리기·저장·재표시가 같은 규칙을 사용하도록 구현했습니다.

구현 구조와 설계

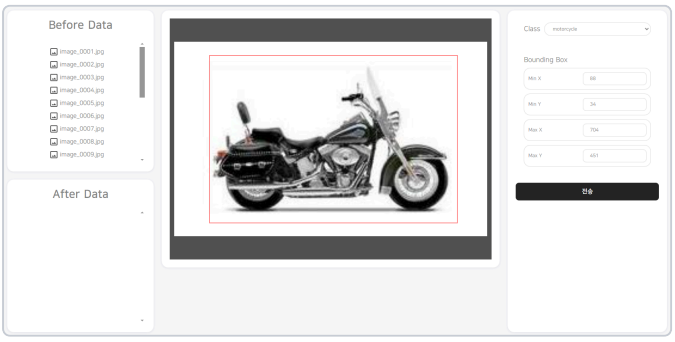
서비스 구성과 프론트엔드 담당 범위

React 프론트엔드는 웹 서버 API를 통해 프로젝트·라벨·모델 데이터를 조회하고 변경합니다.
백엔드와 AI 서버가 연결된 전체 구성에서 프론트엔드 화면과 API 연동을 담당했습니다.
API 호출을 auth·project·class·datamanage·aiModel·operation 등 도메인별 서비스 모듈로 나눴습니다.
축소된 화면에서 그린 좌표는 원본 이미지 좌표로 변환해 저장하고, 다시 표시할 때 역변환했습니다.
양방향 변환은 useCoordinateTransform 훅으로 분리해 그리기·저장·재표시에 같은 규칙을 적용했습니다.

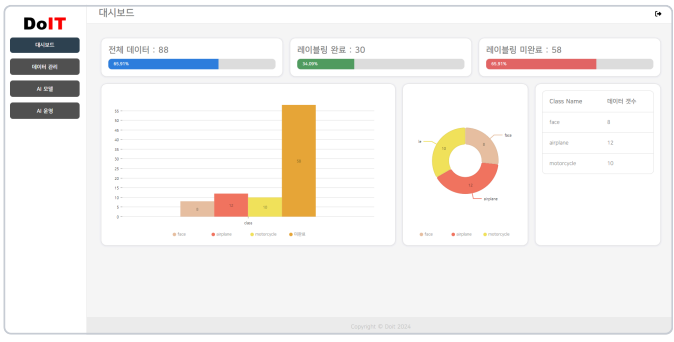


D.A.X 전체 서비스 구성: 담당 범위는 React 프론트엔드와 API 연동

D.A.X · 프로젝트 화면



D.A.X에서 이미지 위에 바운딩 박스를 그려 라벨링하는 화면



D.A.X의 프로젝트 데이터와 학습 현황 대시보드

기술 스택

프론트엔드	TypeScript · React · styled-components · TanStack Query · Zustand

XRGIS 백오피스 XR·GIS 원격 협업 시스템 백오피스

2023.08 – 2023.12 프론트엔드·백엔드 개발 담당

가상·증강·혼합현실(XR)과 지도·공간정보(GIS)를 결합한 원격 협업 서비스입니다.
사용자·팀 관리와 OBJ 모델·영상 파일 처리를 위한 백오피스를 개발했습니다.
React 화면과 Express API를 구현하고 GridFS 기반 파일 저장 및 MariaDB 기반 세션 인증을 연동했습니다.

주요 업무

풀스택 개발	React 기반 사용자·팀·데이터 관리 화면과 Express API 개발
파일 처리	MongoDB GridFS 기반 대용량 파일 업로드, archiver 기반 ZIP 일괄 다운로드 구현
인증·권한	사내 공통 관리자 계정의 세션 인증과 팀 소속에 따른 파일 다운로드 권한 적용
상태·피드백	서버 상태는 React Query, UI 상태는 Recoil로 분리하고 API 결과를 Toast로 안내

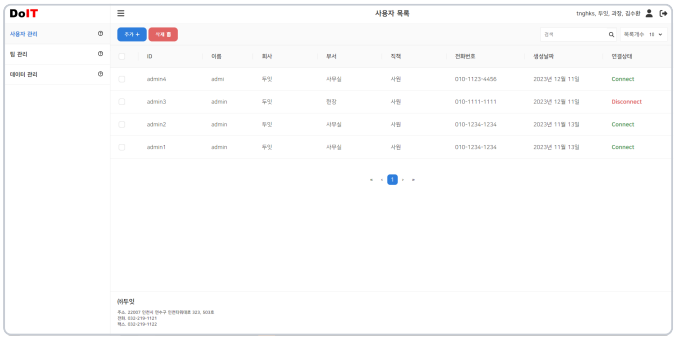
> MongoDB에 파일을 저장해야 하는 요구사항에 맞춰, 일반 문서의 16MB 제한에 걸리는 OBJ·영상 등을 GridFS의 청크 분할 저장으로 처리했습니다.
> 같은 관리자 계정을 사용하는 사내 서비스와는 MariaDB 기반 세션 저장소를 통해 인증 체계를 맞췄습니다.

구현 구조와 설계

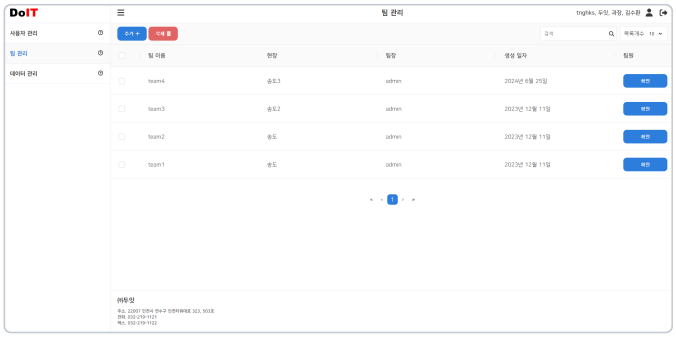
팀별 접근 권한과 대용량 파일 처리

파일 관리 API에서 사용자의 팀 소속에 따라 다운로드 권한을 확인합니다.
같은 관리자 계정을 사용하는 사내 서비스와는 MariaDB 기반 세션 저장소를 통해 인증 체계를 맞췄습니다.
OBJ·영상 파일은 MongoDB GridFS로 청크를 나눠 저장하고, 선택한 파일은 archiver로 ZIP을 생성해 다운로드하도록 구현했습니다.

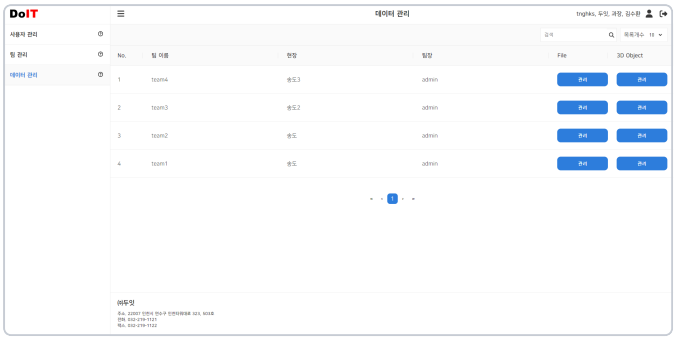
XRGIS 백오피스 · 프로젝트 화면



XRGIS 사용자 관리 화면



XRGIS 팀 관리 화면



XRGIS 팀별 데이터·파일 관리 화면: 대용량 파일 관리와 ZIP 다운로드 기능

기술 스택

프론트엔드	TypeScript · React · styled-components
백엔드	Node.js · Express · MongoDB(GridFS) · MariaDB